

TRACE: 延迟证据视觉运动模仿的轨迹路由因果记忆

李子豪¹邱然鹏³陈银聪任国强¹支伟明² ¹浙江大学²悉尼大学³浙江工业大学

摘要：自主操作的机器人可能需要基于不再可见的证据做出决策。我们研究 *delayed-evidence* 任务，其中早期提示在稍后的决策点之前消失，因此视觉上相似的观察可能需要不同的操作。在这些设置中，当前观察结果不足以进行控制。我们引入了 TRAjectory-routed Causal Evidence (TRACE)，这是一种视觉运动模仿策略的记忆框架。TRACE 将与任务相关的视觉和机器人状态证据（例如对象身份、目标选择或路线相关状态）存储在固定大小的潜在记忆中，该记忆在长时间事件中仍然受到限制。TRACE 不是通过原始时间或手动提供的任务标签来索引内存，而是使用 *path signatures*：执行的机器人状态轨迹的紧凑、顺序敏感的特征。这些签名本身并不存储视觉提示；而是存储视觉提示。相反，它们提供了轨迹条件键，用于写入和检索提示可见时存储的证据。当机器人后来达到模糊的观察结果时，TRACE 内存上的策略条件会恢复丢失的上下文并选择正确的分支。TRACE 通过轻量级适配器将策略附加到策略上，而无需更改策略主干、操作头或模拟目标。在现实世界中具有视觉模糊分支点的长视野操作任务中，TRACE 相对于替代基线（包括短历史和循环记忆）改进了分支选择和任务成功。项目页面：<https://jeong-zju.github.io/trace>

1 简介

机器人通常需要使用不再观察到的信息做出后续任务决策。我们将这些决策称为 *branches*：替代方案，例如下一步要执行哪个目标、路由或操作例程。我们研究 *delayed-evidence manipulation*，其中观察到早期提示，然后消失，并且仅在稍后的决策点需要。到那时，来自不同历史的观察结果可能看起来几乎相同，但需要采取不同的行动。

视觉运动策略的模仿学习 [1, 2] 使得无法直接指定运动规划的成本成为可能。大多数视觉运动策略以当前观察为条件，可能具有较短的最近窗口，因此将此输入视为足以进行动作选择。这是隐式的 *Markov assumption*：当前输入包含选择下一个操作所需的所有信息。延迟证据任务违反了这一假设，因为当前观察并不能确定正确的分支。添加更多历史记录是不够的。一旦决定性线索落在窗口之外，短窗口就会失败，而长窗口会增加成本并迫使政策推断哪些早期观察结果仍然重要。循环策略和通用记忆可以向前传递信息，但与分支相关的线索可能会被覆盖、稀释或与与后续选择无关的任务进度信号纠缠在一起。因此，延迟证据操纵需要一种因果的、固定预算的记忆，该记忆可以在线更新并保留早期任务证据直到它变得有用。

我们引入了 TRAjectory-routed Causal Evidence (TRACE)，这是一种视觉运动策略的记忆框架，其当前的观察不足以进行动作选择。TRACE 将与任务相关的视觉和机器人状态证据存储在一组固定的潜在内存插槽中，并在线更新它们

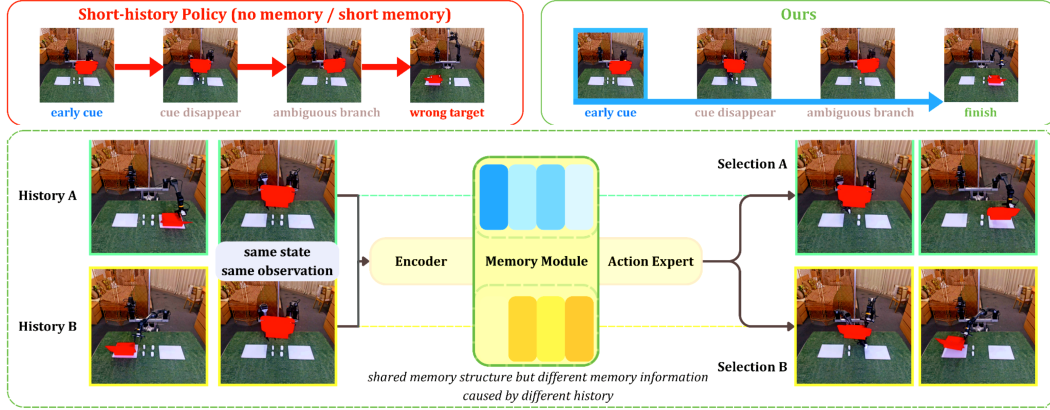


图 1: 长视野操作中的延迟证据: 在分支点, 机器人必须选择一个任务继续。基于过去的观察结果可能看起来很相似, 尽管它们需要不同的行动。短历史策略会失败, 因为它的窗口包含最新信息, 但不包含任何历史线索。TRACE 在提示可见时存储该提示, 并稍后读取该内存以实现正确的选择。

机器人行动。TRACE 不是通过原始时间或任务标签来索引内存, 而是使用 *path signatures* [3], 它们是已执行的机器人状态轨迹的固定长度、顺序敏感的摘要, 作为写入和读取内存的键。这些签名本身并不存储视觉提示; 而是存储视觉提示。相反, 它们提供对线索可见时存储的证据进行轨迹条件访问。这使得 TRACE 可以在不同历史记录中在视觉上相似的分支点检索不同的存储证据。然后, 该策略以 TRACE 内存为条件来恢复丢失的上下文并选择正确的分支。TRACE 通过轻量级适配器附加到策略, 而不改变其主干、动作头或模仿损失。

具体来说, 我们的技术贡献是:

- 延迟证据模仿: 我们制定了一类视觉运动模仿问题, 其中视觉相似的分支点需要不同的动作, 因为决定性的证据出现在事件的早期。
- TRACE内存: 我们引入了一种固定预算因果内存, 它在线存储视觉和机器人状态证据, 并使用执行的机器人状态轨迹的路径签名作为写入和读取内存的顺序敏感键。
- 插件集成和评估: 我们通过轻量级适配器将 TRACE 附加到动作分块和扩散策略, 而不改变其主干、动作头或模仿损失, 并在现实世界的延迟证据操作任务中对其进行评估, 并通过诊断显示从保存的历史证据中获得的收益。

2 相关工作

视觉运动模仿策略和记忆: 现代视觉运动策略将最近的观察结果映射到动作、动作块或动作分布。动作分块策略通过时间聚合来预测短期视野[4], 运动的概率表示能够对不确定性进行推理[5, 6], 而扩散策略通过迭代去噪生成动作序列[7]。基于动态系统的策略是另一种可以保证稳健性的表示形式 [8, 9]。更大的通才政策通过语言调节和广泛的机器人数据集扩展了这一范式[10,11,12,13,14,15,16]。从历史上看, 有关环境的记忆被封装在环境的地图表示中[17, 18]。尽管存在这些差异, 但它们的行为受到调节状态下可用信息的限制。在延迟证据任务中, 当前帧或短观察窗口可能不再包含后续分支所需的线索。循环策略、上下文窗口和显式记忆机制通过传递信息来解决部分可观察性问题, 但可能需要骨干变化、长上下文、演示检索或规划者特定的记忆[19,20,21]。TRACE 改为在线附加固定预算

通过轻量级适配器记忆行动策略，而不改变策略主干、行动头或模仿目标。

轨迹签名和潜在历史表示：路径签名提供了连续轨迹的紧凑的、顺序敏感的摘要，并已被用作序列特征和循环门控机制 [3, 22]。循环策略、变换器上下文窗口和显式记忆系统通过传递信息来解决部分可观察性问题。最近的例子包括层局部外部存储器[19]、冻结策略的检索提示存储器[20]、陈述性场景和情景存储器[21]以及用于分层模仿的关键帧存储器[23]。这些方法改善了时间背景，但通常需要修改策略主干、从演示中检索或为规划者提供专门的记忆。TRACE 以不同的方式使用潜在历史：路径和增量签名不是策略主干、循环隐藏状态或未来预测目标。它们充当轨迹条件键，用于在固定预算存储器中写入和读取视觉和机器人状态证据，将存储器访问与执行的轨迹联系起来，同时保持基本模仿损失不变。

3 问题设置和背景

我们使用 *history* 表示在选择下一个动作之前可用的因果执行信息。

延迟证据模仿：我们研究具有早期提示、共享执行段和后期模糊分支点的模仿任务。

branch 是一种可能的任务延续，例如目标、路线或操作例程，而 *branch point* 是策略必须在这些延续中进行选择的时间。在这一点上，不同的历史可以产生视觉上相似的结果，但需要不同的专家行动，因此仅从当前的观察结果无法解决歧义。

我们考虑演示 $\mathcal{D} = \{\tau_i\}_{i=1}^N$ ，其中 $\tau_i = \{(o_t^i, s_t^i, a_t^i)\}_{t=1}^{T_i}$ 包含多视图观察 o_t 、机器人状态 s_t 和演示动作 a_t 。让 $x_t = (o_t, s_t)$ 和 $\mathcal{H}_t = (x_1, a_1, \dots, x_{t-1}, a_{t-1}, x_t)$ 表示选择 a_t 之前可用的历史记录。在延迟证据任务中， $\mathcal{H}_t$ 可能包含当前观察窗口中不存在的任务相关证据。因此，对于长度为 w 的短窗口，可能存在历史 $\mathcal{H}_t$ 和 $\mathcal{H}'_t$ ，使得

$$x_{t-w+1:t} \approx x'_{t-w+1:t}, \quad a_t^*(\mathcal{H}_t) \neq a_t^*(\mathcal{H}'_t), \quad (1)$$

其中 $a_t^*(\mathcal{H}_t)$ 表示历史记录 $\mathcal{H}_t$ 之后所需的专家目标。我们使用单步动作表示法来设置问题，而我们实现中的策略本机预测目标可能是动作、动作块、动作令牌序列或扩散去噪目标。

路径签名作为流轨迹键：TRACE 需要一个因果记忆键：一个关于机器人如何达到当前状态的紧凑描述符，在线维护而不存储完整的历史记录。给定到时间 t 的机器人状态轨迹的分段线性插值，写为 $S_t : [0, 1] \rightarrow \mathbb{R}^{d_s}$ ，其深度 p 路径签名为

$$\text{Sig}_{\leq p}(S_t) = \left(1, \int dS, \int_{u_1 < u_2} dS_{u_1} \otimes dS_{u_2}, \dots, \int_{u_1 < \dots < u_p} dS_{u_1} \otimes \dots \otimes dS_{u_p} \right). \quad (2)$$

签名总结了状态坐标变化之间的净变化和有序交互，使它们对轨迹如何展开敏感，而不仅仅是它访问的状态[22]。对于连续路径，它们对于单调时间重新参数化是不变的；对于采样的机器人轨迹，分段线性签名给出的描述符通常与时间步索引相比与原始执行速度的关系较小。这为 TRACE 提供了确定性的、固定预算的轨迹密钥，而不是使用另一个学习到的循环状态作为内存和地址。签名还支持流式更新，因此 TRACE 可以在新状态到达时在线维护轨迹密钥。

在 TRACE 中，签名是用于内存访问的确定性轨迹描述符。它们不是任务标签、视觉记忆、循环隐藏状态或策略输出。视觉和机器人状态特征存储任务证据，而签名有助于确定证据的写入位置和

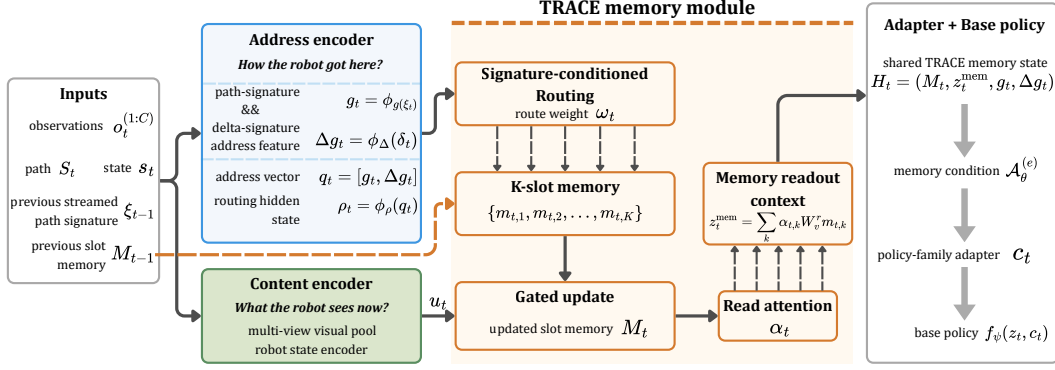


图 2: TRACE 信号流。TRACE 将当前视觉状态证据编码为记忆内容, 使用流式路径签名特征作为轨迹派生键, 更新固定大小的潜在记忆槽, 并将紧凑的记忆条件返回给基本视觉运动策略。

读。我们用 $\xi_t = \text{Sig}_{\leq p}(S_t)$ 表示流式轨迹签名, 用 $\delta_1 = \mathbf{0}$ 表示签名坐标 $\delta_t = \xi_t - \xi_{t-1}$ 中的局部变化特征。我们在附录 A.5 中给出签名尺寸详细信息, 在附录 A.4 中给出机器人状态表示。

4 TRAjectory 路由因果证据 (TRACE)

TRACE 通过固定大小的因果记忆增强了现有的视觉运动模仿策略。在执行过程中, 它将与任务相关的视觉和机器人状态证据写入潜在槽中, 然后在当前观察变得模糊时读取该内存。基本策略保留其原始动作表示、动作头和模仿损失; TRACE 仅通过轻量级适配器添加内存调节路径。图 2 总结了 TRACE 的组件。

4.1 因果记忆接口和轨迹键

令 $z_t = x_{t-w+1:t}$ 为基本策略使用的最近观察窗口, 其中 $x_t = (o_t, s_t)$ 包含多视图观察 o_t 和机器人状态 s_t 。我们将下游策略写为

$$\hat{y}_t = f_\psi(z_t, c_t), \quad (3)$$

其中 $\hat{y}_t$ 是策略的本机预测目标, 例如动作、动作块、动作令牌序列或扩散去噪目标。TRACE 提供 c_t , 这是一种内存条件, 其中包含在 z_t 中可能不再可见的因果信息。

TRACE 将 *memory content* 与 *memory access* 分开。当前图像和机器人状态提供了应该记住的内容, 而执行的轨迹则确定了该信息的存储位置和稍后的检索位置。因此, 签名不是简单地连接到策略输入; 而是简单地连接到策略输入。它们通过一组固定的潜在槽路由内存写入和读取。在每个时间步, 在观察 x_t 之后并选择下一个操作之前, TRACE 将当前证据写入内存, 从更新的内存中读取, 然后查询策略:

$$R_t = (M_t, z_t^{\text{mem}}, g_t, \Delta g_t), \quad c_t = \mathcal{A}_\theta^{(e)}(R_t), \quad \hat{y}_t = f_\psi(z_t, c_t). \quad (4)$$

这里 $M_t \in \mathbb{R}^{K \times d}$ 是 K -slot 潜在内存, z_t^{mem} 是内存读出, g_t 和 Δg_t 是轨迹导出的关键特征, $\mathcal{A}_\theta^{(e)}$ 是策略族 e 的适配器。

为了形成轨迹键, TRACE 使用截至时间 t 的归一化机器人状态路径, 写为分段线性路径 $S_t : [0, 1] \rightarrow \mathbb{R}^{d_s}$ 。它维护一个流式深度 p 路径签名 $\xi_t = \text{Sig}_{\leq p}(S_t)$ 和一个签名空间增量 $\delta_t = \xi_t - \xi_{t-1}$, 以及 $\delta_1 = \mathbf{0}$ 。累积签名 ξ_t 提供全局轨迹地址, 而增量 δ_t 公开同一坐标系中的最近运动, 允许路由依赖于长视野路径上下文和局部进度。学习的投影将它们嵌入为 $g_t = \phi_g(\xi_t)$ 和 $\Delta g_t = \phi_\Delta(\delta_t)$, 并形成轨迹键

$$q_t = \phi_q([g_t, \Delta g_t]). \quad (5)$$

除非另有说明，所有 ϕ 映射都是学习投影或 MLP。关键点是 q_t 是轨迹导出的：它使用有序的机器人状态历史来索引内存访问，而内存内容存储视觉状态证据。

4.2 签名路由槽内存

TRACE 使用轨迹键在固定大小的潜在内存上路由写入和读取。在每一步中，当前的视觉状态输入被编码为 $e_t = \phi_x(x_t)$ ，一个池化的多视图视觉和本体感受特征。此功能包含机器人当前可用的证据，例如对象身份、目标选择或依赖于路线的提示。因此，TRACE 可以在提示可见时存储提示，并在当前观察不再包含该信息时将其公开。

存储器包含槽位 $M_t = \{m_{t,1}, \dots, m_{t,K}\}$ ，每个槽位为 $m_{t,k} \in \mathbb{R}^d$ 。每次演示扫描或在线剧集开始时都会重置插槽。由于路由取决于当前轨迹键和先前的槽内容，因此槽选择可以随着内存内容的演变而调整，而不是遵循轨迹的固定散列。给定轨迹键 q_t ，TRACE 计算路由状态 $\rho_t = \phi_\rho(q_t)$ 并将其与之前的槽内容进行比较：

$$\bar{m}_{t-1,k} = m_{t-1,k} + \lambda_\eta \eta_k, \quad \ell_{t,k} = \frac{(W_Q \rho_t)^\top W_K \bar{m}_{t-1,k}}{\tau \sqrt{d_r}}, \quad \omega_{t,k} = \text{softmax}_k(\ell_{t,k}). \quad (6)$$

这里 η_k 是可选的固定槽标识， λ_η 控制其权重， τ 是路由温度， $\omega_{t,k}$ 决定当前轨迹键选择槽 k 的强度。槽标识在寻址期间打破了初始槽对称性，但不存储为循环存储器内容。

TRACE 通过门控更新将当前证据写入选定的槽中。我们首先形成一个写入输入 $u_t = \phi_w(e_t, q_t)$ ，它将当前的视觉状态证据与轨迹键结合起来。然后插槽更新为

$$\tilde{m}_{t,k} = \tanh(\phi_m(\bar{m}_{t-1,k}, u_t, \rho_t)), \quad \beta_{t,k} = \omega_{t,k} \sigma(\phi_\beta(\bar{m}_{t-1,k}, u_t, \rho_t)), \quad (7)$$

$$m_{t,k} = (1 - \beta_{t,k}) m_{t-1,k} + \beta_{t,k} \tilde{m}_{t,k}. \quad (8)$$

这里 $\tilde{m}_{t,k}$ 是插槽 k 的建议新内容， $\beta_{t,k}$ 是路由调制的写门。小 $\omega_{t,k}$ 的槽几乎保持不变，而选定的槽吸收当前视觉状态证据。写入后，TRACE 使用由当前证据和轨迹状态形成的查询从更新的槽中读取：

$$\alpha_{t,k} = \text{softmax}_k \left(\frac{\phi_r(e_t, \rho_t)^\top W_K^r (m_{t,k} + \lambda_\eta \eta_k)}{\sqrt{d}} \right), \quad z_t^{\text{mem}} = \sum_{k=1}^K \alpha_{t,k} W_V^r m_{t,k}. \quad (9)$$

这里 $\alpha_{t,k}$ 是读取权重， W_K^r, W_V^r 是学习的读取键和读取值投影。共享的 TRACE 状态是 $R_t = (M_t, z_t^{\text{mem}}, g_t, \Delta g_t)$ 。这里 $M_t \in \mathbb{R}^{K \times d}$ 是更新后的 K 槽潜在内存， z_t^{mem} 是从 M_t 读取的数据， g_t 和 Δg_t 是轨迹导出的关键特征， $\mathcal{A}_\theta^{(e)}$ 是策略族 e 的适配器。完整的投影、辅助损失和伪代码详细信息在附录 A.9 中给出。

训练和部署都使用相同的因果扫描：TRACE 仅接收策略查询之前可用的观察结果和机器人状态，并且不使用提示标签、分支标签、未来帧或测试时演示检索。在每一步中，内存更新仅使用截至该时间可用的观察和状态，并将结果条件 c_t 传递给基本策略的未更改的模仿损失。

4.3 策略适配器：从TRACE内存到策略调节

TRACE 更新程序在策略系列之间共享；仅面向策略的适配器发生变化。给定共享 TRACE 状态 $R_t = (M_t, z_t^{\text{mem}}, g_t, \Delta g_t)$ ，适配器将内存映射到策略族 e 的本机调节空间中，作为 $c_t^{(e)} = \mathcal{A}_\theta^{(e)}(R_t) \in \mathcal{C}_e$ 。基本策略的动作表示、动作头和监督模仿损失保持不变。

For action-chunking regression policies, the adapter projects the memory slots into 512-D memory tokens and maps $[z_t^{\text{mem}}, g_t, \Delta g_t]$ into a summary token. These tokens are concatenated to the policy's attention memory, after which the original action head regresses the native action chunk. 对于扩散策略，适配器使用读取权重对槽进行池化，将池化内存与 z_t^{mem} 、 g_t 和 Δg_t 连接起来，并通过零初始化的MLP将结果映射到加法全局调节向量中。The denoising process and diffusion on loss are otherwise unchanged.

因此，适配器只是从 TRACE 内存到策略现有调节接口的转换器，而不是新的策略主干、操作解码器或检索模块。Appendix A.8 gives architectural details, parameter counts, latency, and training details.

5 实证评估

We organise the experiments around four questions: (1) Does adding causal memory improve delayed-evidence manipulation? (2) Does the gain transfer across policy families? (3) Does TRACE improve over generic history memory? (4) Does TRACE use historical evidence rather than relying only on the decision-point observation? 我们通过基本策略比较、跨策略 TRACE 变体、内存消融和历史转换诊断来回答这些问题。

实验设置。 Figure 3 illustrates the delayed-evidence task suite. We collect data with the system described in [24]. We evaluate on five real-world tasks: *Tool*, where initial object identity determines a later tool sequence; *Book*, where origin determines route and placement; *Laundry*, where origin side determines brush-and-basket versus fold-and-store; *Cable*, where origin side determines the matched device; and *Medicine*, where tray origin determines box selection and return. Appendix A.3 gives task, dataset, and rollout details.

基线。 我们将 TRACE 附加到回归式动作分块策略和基于扩散的策略，并与相应的基本策略进行比较。我们还评估了用于模仿或语言条件控制的可部署机器人策略系列，包括 SmoVL A [14]、 $\pi_{0.5}$ [13]、X-VLA [15] 和 GR00T N1.6 [16]。These VLA baselines are not frozen zero-shot models: each is adapted on the same single-task demonstrations and train/evaluation split. Appendix A.1 gives the model setups, and Appendix A.2 reports training hyperparameters and parameter counts for all baselines and TRACE modules.

指标。 在长范围操作基准 [25, 26] 中的部分进度报告之后，我们通过有序子任务的阶段级进度对每次推出进行评分，包括操作阶段和依赖于内存的分支决策。We report task-balanced averages, rollout standard errors, bootstrap uncertainty, and full scoring details in Appendix A.1.

问题 1. 记忆有助于延迟证据操纵吗？是的。表 1 比较了基本视觉运动策略、微调的 VLA 型基线以及使用 TRACE 因果记忆增强的相同基本策略系列。TRACE 显著改进了这两个策略系列：回归平均进度从 25.50 上升到 69.23，扩散从 25.00 上升到 59.53。TRACE 的回归性能也优于最强的非 TRACE 基线 $\pi_{0.5}$ ，高出 18.76 点，扩散性能高出 9.06 点。最大的收获发生在早期观察到的原点或物体线索确定线索离开视野后的后续分支的任务中。



图 3：所选延迟证据操作任务的概述。

外卖。 TRACE 改进了对没有因果记忆的政策延迟证据操纵，在书籍、洗衣和医学方面的收益最明显，其中早期的起源线索决定了后来视觉上模糊的分支。总体结果不仅仅由高方差工具任务驱动：附录 A.11 报告了留一任务和特定于工具的分析。附录 A.10

表 1: 现实世界延迟证据任务的主要比较。值是平均阶段进度 (%) $\pm$ SE。

Method	Tool $\uparrow$	Book $\uparrow$	Laundry $\uparrow$	Cable $\uparrow$	Medicine $\uparrow$	Avg. progress $\uparrow$
Diffusion Policy	18.00 $\pm$ 1.41	12.33 $\pm$ 0.44	22.00 $\pm$ 0.32	34.00 $\pm$ 0.48	38.67 $\pm$ 1.01	25.00 $\pm$ 0.38
ACT	31.00 $\pm$ 5.51	13.67 $\pm$ 0.49	11.50 $\pm$ 2.01	44.00 $\pm$ 7.79	27.33 $\pm$ 0.05	25.50 $\pm$ 1.95
$\pi_{0.5}$	45.50 $\pm$ 0.81	51.00 $\pm$ 6.99	47.50 $\pm$ 1.45	49.00 $\pm$ 0.57	59.33 $\pm$ 1.45	50.47 $\pm$ 1.47
GR00T N1.6	39.00 $\pm$ 2.31	12.67 $\pm$ 2.14	24.00 $\pm$ 2.72	38.00 $\pm$ 12.11	39.33 $\pm$ 3.29	30.60 $\pm$ 2.64
SmolVLA	25.00 $\pm$ 1.51	20.00 $\pm$ 0.82	12.00 $\pm$ 2.16	50.00 $\pm$ 1.44	34.67 $\pm$ 1.01	28.33 $\pm$ 0.65
X-VLA	5.50 $\pm$ 0.94	47.00 $\pm$ 6.19	41.00 $\pm$ 0.36	36.00 $\pm$ 21.44	40.00 $\pm$ 2.92	33.90 $\pm$ 4.51
Ours (Regression)	54.50 $\pm$ 17.96	83.00 $\pm$ 0.86	81.00 $\pm$ 2.84	51.00 $\pm$ 1.11	76.67 $\pm$ 1.05	69.23 $\pm$ 3.65
Ours (Diffusion)	58.50 $\pm$ 10.17	68.33 $\pm$ 2.86	70.50 $\pm$ 0.76	51.00 $\pm$ 4.63	49.33 $\pm$ 0.17	59.53 $\pm$ 2.31

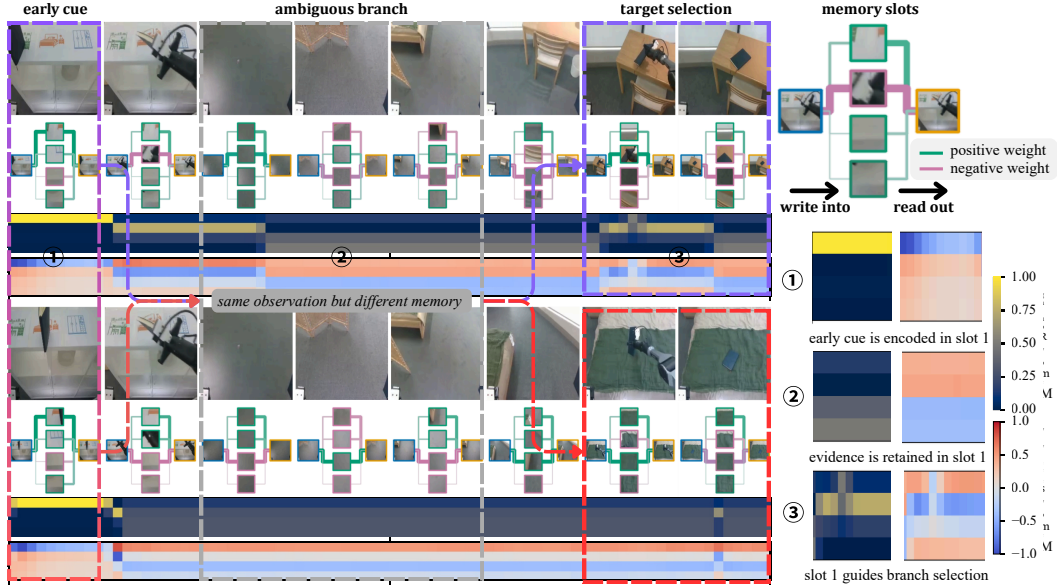


图 4: *Book* 的推出结果。时间线包含过去的提示、视觉上模糊的传输和目标选择，而重叠的槽图和右侧面板显示证据被写入、保留和读取的位置。正数和负数表示带符号的内存权重：正权重增加对所选插槽的支持，而负权重则带有相反符号的证据来抑制这些插槽。

进一步将常见分支错误、推出证据和 TRACE 行为与任务案例联系起来。部署开销在附录 A.8 中测量。

问题 2. 收益是否仅限于特定保单系列？不。表 1 显示 TRACE 在使用训练中使用的相同更新器、相同签名路由和相同历史扫描的同时改进了回归和扩散变体。只有面向策略的适配器发生变化：回归使用适配器作为动作分块回归器，而扩散使用适配器作为扩散策略。在这些测量的推出中，平均回归更强。它的记忆条件在一次前向传递中输入，而扩散采样器则通过重复的细化来携带该条件。重要的一点是，两个策略系列都受益于相同的 TRACE 内存状态。这支持了模块化设计的主张，即 TRACE 提供缺失的历史信息，同时保持下游模仿目标和动作头完好无损。

图 4 显示了测得的内存信号。签名索引写入不会折叠到一个槽。路由随情节进展而变化，而读数仍然选择性地选择在分支点附近携带历史证据的时隙。在“Book”卷展栏中，俯视图在拾取和传输后在视觉上变得相似，但内存图通过签名签名分数和高权重槽边缘保留了与原点相关的轨迹。附录 A.10 提供了额外的任务级案例研究，将这些诊断与具体的推出失败和恢复联系起来。

问题 3. TRACE 比通用历史内存更好吗？是的，当延迟的线索必须通过执行的因果历史来保留时。表 2 将 TRACE 与循环、历史标记、外部存储器和检索替代方案在相同的回归基本策略、标准化下进行了比较

表 2: 内存模块比较。循环、变压器上下文、LRU 和检索提示控件的灵感来自于 GRU 门控 [27]、自注意力上下文 [28]、最近最少使用的外部内存访问 [29] 和内存增强提示 [20]。值是平均阶段进度百分比 $\pm$ SE。

Memory module	Online	Fixed	Sig.	Tool	Book	Laundry	Cable	Medicine	Avg.
No memory	–	–	–	31.00 $\pm$ 5.51	13.67 $\pm$ 0.49	11.50 $\pm$ 2.01	44.00 $\pm$ 7.79	27.33 $\pm$ 0.05	25.50 $\pm$ 1.95
GRU recurrent memory	✓	✓	–	40.50 $\pm$ 9.12	49.00 $\pm$ 3.04	44.00 $\pm$ 2.21	47.00 $\pm$ 4.98	48.67 $\pm$ 1.24	45.83 $\pm$ 1.91
Transformer history context	–	–	–	40.50 $\pm$ 7.83	61.67 $\pm$ 2.54	55.00 $\pm$ 1.96	46.00 $\pm$ 3.87	57.33 $\pm$ 1.41	52.10 $\pm$ 1.68
LRU external memory	✓	✓	–	46.50 $\pm$ 7.64	57.67 $\pm$ 2.48	54.50 $\pm$ 2.03	51.00 $\pm$ 3.52	60.00 $\pm$ 1.22	53.93 $\pm$ 1.61
Retrieval-prompt memory	–	–	–	48.00 $\pm$ 7.66	62.67 $\pm$ 2.18	59.50 $\pm$ 1.77	50.00 $\pm$ 3.62	61.33 $\pm$ 1.12	56.30 $\pm$ 1.46
TRACE signature-routed slots	✓	✓	✓	54.50 $\pm$ 17.96	83.00 $\pm$ 0.86	81.00 $\pm$ 2.84	51.00 $\pm$ 1.11	76.67 $\pm$ 1.05	69.23 $\pm$ 3.65

表 3: 消融和历史转换诊断。路由相似性衡量转换后的在线推出历史与原始 Full TRACE 推出的槽路由序列之间的一致性。分支一致性衡量分支点存储器读出和分支操作之间的一致性。两种诊断均标准化为 Full TRACE，仅用于跨任务比较。

Component ablations		History-transform diagnostics			
Variant	Avg. $\uparrow$	History transform	Tested property	Route similarity $\uparrow$	Branch consistency $\uparrow$
Current observation only	25.50 $\pm$ 1.95	Time resampling	time-change inv.	92.40 $\pm$ 1.10	96.00 $\pm$ 0.80
Signature-only	45.50 $\pm$ 1.82	Speed jitter	time-change inv.	89.80 $\pm$ 1.40	94.70 $\pm$ 1.00
Unrouted slot memory	52.17 $\pm$ 1.63	State offset	offset inv.	91.20 $\pm$ 1.20	95.10 $\pm$ 0.90
No-delta routing	61.43 $\pm$ 2.21	Sparse sampling	sampling robust.	86.50 $\pm$ 1.60	92.30 $\pm$ 1.20
Mean readout	62.80 $\pm$ 2.44	Order reversal	negative control	37.80 $\pm$ 2.50	41.60 $\pm$ 2.20
No auxiliary losses	66.10 $\pm$ 2.84	Order-preserving controls	summary	89.98 $\pm$ 0.68	94.53 $\pm$ 0.49
Full TRACE	69.23 $\pm$ 3.65	Full TRACE	reference	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00

化和评估块。这些控制询问延迟证据操纵是否只需要更多历史，或者历史是否必须围绕已执行的因果历史进行组织。

外卖。相对于无内存，通用内存有帮助，但表 2 显示上下文长度或存储容量本身并不匹配 TRACE。匹配的控件可以保留历史记录，但它们不会显式地将延迟提示绑定到将在分支点读取的执行轨迹地址。附录 A.10 将这种区别与任务级分支错误和 TRACE 恢复联系起来。

问题 4. 诊断是否表明 TRACE 使用早期证据而不仅仅是决策点的观察结果？是的。表 3 报告了测试 TRACE 是否使用已执行历史记录的消息和历史转换诊断。路由相似性衡量转换后的在线转出历史记录是否通过与分支点之前的原始 TRACE 转出相同的槽路由进行写入。分支一致性衡量分支点读数是否仍然带有相同的延迟分支选择。时间重采样、速度抖动、状态偏移和稀疏采样应该保留相关的历史证据，而顺序反转则故意破坏历史的因果顺序。我们通过同一任务上相应的完整跟踪值对每个原始诊断进行标准化，以实现跨任务可比性。附录 A.6 给出了完整的公式和平均值。

外卖。表 3 支持预期行为。保序变换使路由相似性和分支一致性接近完整 TRACE 参考，而顺序反转则大幅降低这两种诊断。这种对比将记忆读数与有序的历史证据联系起来，而不是与决策点不变观察联系起来。

6 局限性和结论

限制：主要限制是签名的维度。对于标准截断签名，原始特征维度随着学习压缩之前的机器人状态维度和签名深度而快速增长。这会增加归一化、投影、内存和计算成本，并且在需要更高维度的状态或更深的签名时可能会降低 TRACE 的效率。

结论：TRACE 为视觉运动机器人策略提供了执行历史的紧凑因果记忆状态。通过使用路径和增量签名路由视觉状态写入，它将过去的任务证据存储在固定大小的 TRACE 内存状态中，并通过轻量级适配器呈现该内存。这产生了一种模块化设计，其中内存和动作生成与

基本政策目标。在五个现实世界的延迟证据任务中，相同的记忆模块改善了回归和扩散策略系列，并且诊断表明，收益来自记住历史，而不是仅依赖于决策点附近可见的线索。

参考

- [1] H. Ravichandar, A. S. Polydoros, S. Chernova 和 A. Billard。机器人从演示中学习的最新进展。 *Annual review of control, robotics, and autonomous systems*, 2020。
- [2] W.zhi, T.Lai, L.Ott 和 F.Ramos。用于广义模仿学习的微分同胚变换。 2022 年 *Learning for Dynamics and Control Conference, L4DC*。
- [3] I. Chevyrev 和 A. Kormilitzin。机器学习中的签名方法入门。在 *Signature Methods in Finance: An Introduction with Computational Applications* 中，第 3-64 页。施普林格，2025。
- [4] T.Z.Zhao, V.Kumar, S.Levine 和 C.Finn。使用低成本硬件学习细粒度双手操作。 *arXiv preprint arXiv:2304.13705*, 2023。 [5] W.Zhi, T.Zhang 和 M.Johnson-Roberson。通过素描指导机器人：通过概率图解教学从演示中学习。在 *IEEE International Conference on Robotics and Automation (ICRA)*, 2024 年。
- [6] A. Paraschos, C. Daniel, J. Peters 和 G. Neumann。概率运动原语。在 *Proceedings of the 26th International Conference on Neural Information Processing Systems*, 2013 年。
- [7] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake 和 S. Song。扩散政策：通过行动扩散进行视觉运动政策学习。 *The International Journal of Robotics Research*, 44(10-11): 1684–1704, 2025。
- [8] W.zhi, T.Lai, L.Ott, E.V.Bonilla 和 F.Ramos。通过可逆神经网络学习高效且鲁棒的常微分方程。在 *International Conference on Machine Learning, ICML*, 2022 年。
- [9] W.zhi, H.Tang, T.Zhang, M.Johnson-Roberson。通过草图教学周期性稳定机器人运动生成。 *IEEE Robotics and Automation Letters*, 2025 年。
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu 等人。Rt-1：用于大规模现实世界控制的机器人变压器。 *arXiv preprint arXiv:2212.06817*, 2022 年。
- [11] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid 等。Rt-2：视觉-语言-动作模型将网络知识转移到机器人控制。在 *Conference on Robot Learning* 中，第 2165-2183 页。PMLR, 2023。
- [12] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain 等人。Open X-embodiment：机器人学习数据集和 RT-X 模型。在 *2024 IEEE International Conference on Robotics and Automation (ICRA)* 中，第 6892–6903 页。IEEE, 2024。
- [13] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang 和 U. zhilinsky。 π_0 ：用于通用机器人控制的视觉-语言-动作流模型。 *arXiv preprint arXiv:2410.24164*, 2024 年。

- [14] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Arac tingi, C. Pascal, M. Russi, A. Marafioti 等人。Smolvla: 一种用于经济实惠且高效的机器人技术的视觉-语言-动作模型。 *arXiv preprint arXiv:2506.01844*, 2025 年。
- [15] 郑家, 李家, 王正, 刘东, 康旭, 冯玉, 郑玉, 邹家, 陈玉, 曾家, 等。X-vla: 软提示变压器作为可扩展的跨实施视觉-语言-动作模型。 *arXiv preprint arXiv:2510.10274*, 2025 年。
- [16] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang 等。Gr00t n1: 通用人形机器人的开放基础模型。 *arXiv preprint arXiv:2503.14734*, 2025。[17] W.Zhi, L.Ott, R.Senanayake 和 F.Ramos。连续占用图与快速贝叶斯希尔伯特图融合。 *International Conference on Robotics and Automation (ICRA)*, 2019 年。[18] W.zhi, R. Senanayake, L. Ott 和 F. Ramos。利用核循环混合密度网络对城市环境中的方向不确定性进行时空学习。 *IEEE Robotics and Automation Letters*, 2019。
- [19] E. Cherepanov, A. K. Kovalev 和 A. I. Panov。ELMUR: 具有更新/重写功能的外层存储器, 可解决长期强化学习问题。 *arXiv preprint arXiv:2510.07151*, 2025 年。
- [20] 李瑞, 郭伟, 吴子, 王成, 邓华, 翁正平, Y.-P. 谭和Z.王。MAP-VLA: 机器人操作中视觉-语言-动作模型的记忆增强提示。 *arXiv preprint arXiv:2511.09516*, 2025 年。
- [21] 林明, 梁X, 林B, L.静芝, 焦子, K.李, Y.马, Y.刘, S.赵, Y.庄, 等。EchoVLA: 具有用于移动操作的协同陈述性记忆的机器人视觉-语言-动作模型。 *arXiv preprint arXiv:2511.18112*, 2025 年。
- [22] P.基德和T.莱昂斯。Signatory: 在 CPU 和 GPU 上对签名和 logsignature 变换进行可微分计算。 *arXiv preprint arXiv:2001.00706*, 2020。
- [23] T. Buamane, M. Kobayashi 和 Y. Uranishi。Bi-HIL: 基于双边控制的多模式分层模仿学习, 通过子任务级进度和关键帧内存进行长水平接触丰富的机器人操作。 *arXiv preprint arXiv:2603.13315*, 2026 年。
- [24] 李Z., 周Y., 邱R., 吴H.吴, G.任, W.Zhi。Tripilot-ff: 具有力反馈的协调全身遥控操作。 *arXiv preprint arXiv:2602.09888*, 2026 年。
- [25] M. Heo, Y. Lee, D. Lee 和 J. J. Lim。Furniturebench: 可重复的现实世界基准, 用于长视野复杂操作。 *The International Journal of Robotics Research*, 44 (10-11): 1863-1891, 2025。
- [26] O. Mees, L. Hermann, E. Rosete-Beas 和 W. Burgard。Calvin: 长视野机器人操作任务的语言条件策略学习的基准。 *IEEE Robotics and Automation Letters*, 7(3): 7327-7334, 2022。
- [27] K. Cho, B. Van Merriënboer, C. Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk 和 Y. Bengio。使用 rnn 编码器-解码器学习短语表示以进行统计机器翻译。 *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 第 1724-1734 页, 2014 年。
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, 等。凯撒和波洛苏欣。您所需要的就是关注。 *Advances in neural information processing systems*, 2017 年 30 日。
- [29] A. Santoro, S. Bartunov, M. Botvinick, D. Wierstra 和 T. Lillicrap。使用记忆增强神经网络进行元学习。在 *International conference on machine learning* 中, 第 1842-1850 页。PM LR, 2016。

技术附录

本附录收集了支持正文的技术材料。这些小节遵循论文的叙述。他们定义了 TRACE 的延迟证据设置，指定状态表示，总结训练超参数，陈述签名和记忆方程，定义诊断，并给出适配器、方差和案例研究详细信息，这些细节对于主论文来说太长了。

A.1 问题设置详细信息

我们考虑演示 $\mathcal{D} = \{\tau_i\}_{i=1}^N$

$$\tau_i = \{(o_t^i, s_t^i, a_t^i)\}_{t=1}^{T_i}, \quad x_t^i = (o_t^i, s_t^i), \quad \mathcal{H}_t^i = (x_1^i, a_1^i, \dots, x_{t-1}^i, a_{t-1}^i, x_t^i). \quad (10)$$

这里 x_t 是当前的多视图观察和机器人状态， $\mathcal{H}_t$ 是在时间 t 选择动作之前可用的因果历史。延迟证据任务包含在某些 $t_e < t_b$ 的历史记录中观察到的过去证据变量 $e_i = \eta(\mathcal{H}_{t_e}^i)$ 。该变量可以是原始货架、服装侧、源托盘、对象标识或确定后续分支的另一个提示。在分支步骤 $t = t_b$ ，分支动作 a_t 是其正确值取决于较早证据的动作。

核心的歧义在于，不同的过去证据可能导致在分支点处几乎相同的当前观察结果。 $e_i \neq e_j$ 可以有两条轨迹，使得

$$x_{t_b}^i \approx x_{t_b}^j, \quad a_{t_b}^{i,*} \neq a_{t_b}^{j,*}, \quad p(a_{t_b}^* | \mathcal{H}_{t_b}^i) \neq p(a_{t_b}^* | \mathcal{H}_{t_b}^j). \quad (11)$$

$\pi(a_t | x_t)$ 形式的策略必须为这些不明确的分支观察分配一个操作分布，因此对于此设置来说是不够的。所需的对象是一个紧凑的因果历史摘要 $m_t = m(\mathcal{H}_t)$ ，以便 $\pi(a_t | x_t, m_t)$ 可以恢复依赖证据的分支决策，同时保持接口固定大小。

TRACE 使用在线签名路由槽存储器和从中派生的适配器条件来实现 m_t 。过去的证据变量 e_i 只是一个用于描述问题的未观察到的变量。TRACE 不接收 e_i 作为受监督标签。它也不会接收分支标签、任务标识符、未来观察或未来操作。在训练和部署期间，其内存更新在时间 t 时仅读取 $\mathcal{H}_t$ 中包含的因果历史记录。

每次部署都是通过将任务分解为有序的子任务然后分解为阶段来评分的。该指标的灵感来自于长期操作基准中的部分进度报告，包括 FurnitureBench 中已完成的阶段或子任务以及 C-ALVIN 式评估中的成功任务链长度 [25, 26]。它遵循报告惯例，而不是重复使用相同的公式。对于任务 q ，报告的进度为

$$\text{Progress}_q = \frac{100}{N_q T_q S_q} \sum_{i=1}^{N_q} \sum_{k=1}^{T_q} \sum_{j=1}^{S_q} \mathbf{1}[\text{stage}_{i,k,j} \text{ succeeds}]. \quad (12)$$

这里， N_q 是主评估中的 25 个物理部署， T_q 是子任务的数量， S_q 是每个子任务的阶段数量。总平均值是任务平衡的。标准错误在每个任务中使用推出级引导重采样。

基线控制。表 4 给出了实验中引用的基线方案。比较保持了相机、本体感觉、动作率、列车分割标准化，并提供了跨方法匹配的评估列表。

表 4：外部 VLA 基线协议。语言条件模型仅接收通用任务提示。

Baseline	Initialisation and trainable parts	Budget	Interface controls
SmolVLA	1erobot/smolvla_base, tuned adapter and state mapper, frozen vision	180k updates	3 RGB views, 17-D state, 50-step action chunks
$\pi_{0.5}$	1erobot/pi05_base with the released policy adaptation path	180k updates	224px RGB views, 17-D state, 50-step action chunks
X-VLA	1erobot/xvla_base, tuned policy transformer and soft prompts	180k updates	Same views and state, 16-step action chunks, no cue prompt
GR00T N1.6	Released N1.6 adaptation with tuned embodiment and action head	180k updates	Same rollout API, action rate, and held out episodes

包含基线协议表是为了使比较可审核，而不是添加另一个性能声明。重要的控制是每个基线看到相同的摄像机组流，亲

感知布局、行动率和保留的评估列表。语言条件方法接收通用任务提示，因此延迟的提示不会通过文本泄漏。

A.2 模型大小和超参数

据报道，参数核算和训练超参数支持跨策略系列的可重复性和容量公平性。它们不用作独立的性能声明。表 5 将本地检查点和模型缓存审核的总容量、可训练容量和冻结容量分开。表 7 总结了训练计划和优化器设置，而决定 TRACE 模块大小的架构设置在表 10 中列出。

表 5: 来自本地检查点和模型缓存审核的基线和 TRACE 模块的模型大小摘要。

Model or module	Count scope	Total parameters	Trainable parameters	Frozen parameters or frozen parts	Notes
ACT	Full policy used as the regression base expert	51.62M	51.62M	0	Same observations, state, action rate, and train split as TRACE Regression runs
Diffusion Policy	Full policy used as the diffusion base expert	270.96M	270.96M	0	Same observations, state, action rate, and train split as TRACE Diffusion runs
SmolVLA	Fine tuned LeRobot baseline	450.05M	99.88M	350.17M VLM frozen	Uses the released fine tuning head and matched single task data
$\pi_{0.5}$	Fine tuned LeRobot baseline	3.62B	693.42M	2.92B PaliGemma VLM frozen	Uses the released adaptation path and matched single task data
X-VLA	Fine tuned VLA baseline	879.74M	879.74M	0	Tuned VLM, policy transformer, and soft prompts, with no cue prompt
GR00T N1.6	Fine tuned humanoid policy baseline	2.72B	1.07B	1.66B language/vision frozen	Tuned embodiment interface and action head under the matched rollout API
TRACE updater	Shared signature routed memory updater only	11.91M	11.91M	0	Excludes the base expert and policy specific adapter
Regression adapter	TRACE adapter for the regression base expert	0.79M	0.79M	0	Excludes the shared updater and the base expert
Diffusion adapter	TRACE adapter for the diffusion base expert	16.03M	16.03M	0	Excludes the shared updater and the base diffusion expert

审核将模型容量与内存模块分开。ACT 和 Diffusion Policy 是两个 TRACE 接口的基础专家，而 VLA 基线包括其发布的适应路径和冻结组件。TRACE 更新器在适配器之间共享，因此增加的容量可以归因于固定内存模块加上小型面向策略的转换器，而不是新的操作主干。

表 6 报告了两个面向策略的接口的 TRACE 附加参数预算。参考基础专家仅用作百分比的审核分母。行名称描述了 TRACE 接口而不是固定的组合方法。添加的计数包括共享更新程序和特定于策略的适配器。该百分比根据审核计数计算为 $100 \times N_{\text{added}}/N_{\text{base}}$ 。

表 6: 面向策略的接口的附加 TRACE 参数预算。

TRACE interface	Reference base expert	Added TRACE parts	Base parameters	Added parameters	Increase	Notes
Regression-policy interface	ACT audit checkpoint	Shared updater plus regression-interface adapter	51.62M	12.69M	24.6%	Interface only
Diffusion-policy interface	Diffusion Policy audit checkpoint	Shared updater plus diffusion-interface adapter	270.96M	27.94M	10.3%	Interface only

添加的参数表显示，两个接口由于条件不同的基础专家而具有不同的容量成本。回归接口在审核的 ACT 检查点上添加了 12.69M 参数，扩散接口在审核的扩散策略检查上添加了 27.94M -

观点。这些计数用于容量核算。它们不用来替代表 1 中的物理部署比较。

表 7: 所报告协议使用的训练超参数。所有策略均经过 180k 次更新训练；批量大小和操作窗口遵循每个策略系列。

Policy family	Code policy	Updates / batch	Action window	Optimiser and scheduler	Fine-tuning settings
ACT regression	act	180k updates; batch 8 or 32	Chunk 100, execute 100	LeRobot ACT optimiser preset; AMP enabled	No TRACE memory; same cameras, state, action normalisation, train split, and evaluation API as TRACE Regression
Diffusion Policy	diffusion	180k updates; batch 8	2 obs. steps, horizon 104, execute 100, drop last 3 frames	LeRobot Diffusion optimiser preset; AMP enabled	No TRACE memory; same observation and action interface as TRACE Diffusion
SmolVLA	smolvla	180k updates; batch 8 or 16	1 obs. step, chunk 50, execute 50, 10 flow steps	AdamW, lr 10^{-4} , weight decay 10^{-10} , grad clip 10; 1000 warmup steps and cosine decay over 30k steps to 2.5×10^{-6}	Frozen vision encoder; train action expert and state projection; generic task prompt only
$\pi_{0.5}$	pi05	180k updates; batch 1	1 obs. step, chunk 50, execute 50, 10 inference steps	AdamW, lr 2.5×10^{-5} , weight decay 0.01, grad clip 1; 1000 warmup steps and cosine decay over 30k steps to 2.5×10^{-6}	Frozen vision encoder; train action expert branch; mean/std state and action normalisation for the Zeno datasets
X-VLA	xvla	180k updates; batch 1	1 obs. step, chunk 16, execute 16, 10 denoising steps	AdamW, lr 10^{-4} , weight decay 0, grad clip 10; 1000 warmup steps and cosine decay over 30k steps to 2.5×10^{-6}	Train policy transformer and soft prompts; no cue prompt
GR00T N1.6	External adaptation	180k updates	Same rollout API and action rate as the robot policies	Released N1.6 adaptation recipe	Tune embodiment interface and action head; language and vision components frozen as in the parameter audit

训练表有意与下面的架构表分开。表 7 记录了报告的训练协议中使用的优化预算、行动范围和微调选择。表 10 记录了决定 TRACE 模块大小和路由接口的内存架构值。

A.3 数据集和演示详细信息

表 8 明确了五个真实机器人任务的数据统计。物理推出计数是公式 12 中使用的主要评估计数。工具、书籍和洗衣计数和范围是根据处理后的 LeRobot 元数据进行审核的。使用与转换器相同的 30 Hz 采样规则从本地 ROS 包时间戳扫描审核电缆和医学计数和范围。

表 8: 本文报告的真实机器人评估协议的数据集规模和 SS 详细信息。

Task	Train demos	Validation demos	Physical eval. rollouts	Episode length or avg. horizon	Subtasks / stages	Held-out factors
Tool	100	0	25	2,910 frames (97.0s)	2/4	Object instances, cue assignments, poses, lighting, distractors
Book	150	0	25	1,392 frames (46.4s)	3/4	Origin cue assignments, poses, routes, lighting, distractors
Laundry	60	0	25	2,220 frames (74.0s)	2/4	Garment instances, origin side assignments, poses, lighting, distractors
Cable	30	0	25	1,770 frames (59.0s)	1/4	Origin side assignments, device poses, lighting, distractors
Medicine	60	0	25	1,759 frames (58.6s)	2/4	Tray origin assignments, box poses, lighting, distractors, bedside props

数据集表使证据来源明确。每项任务都使用 25 个身体评估卷展，主表中报告的进度分数是根据这些卷展计算得出的。列车演示计数和范围来自经过审核的元数据或 ROS 包时间戳扫描。保留因素表明，评估会改变对象实例、提示分配、姿势、照明和干扰因素，而不是重复训练片段。

A.4 状态表示和标准化

策略主干和 TRACE 使用一个头顶 RGB 摄像头、两个腕部 RGB 摄像头和 17-D 本体感受状态。该状态被排序为基础平面速度、基础偏航速度、左臂关节位置和右臂关节位置。固定基础任务保持相同的布局，并在签名计算之前将基础通道设置为零。零掩码在列车分割标准化之前应用，因此恒定通道不会贡献路径增量。

在每个控制步骤，屏蔽状态被标准化并附加到流式历史路径中。第一个观察到的状态用作基点。TRACE 省略标量签名坐标。对于 $d_s=17$ 和 $p=3$ ，这给出 $d_{\text{sig}} = 17 + 17^2 + 17^3 = 5,219$ 。在学习映射 ϕ_g 和 ϕ_Δ 之前，路径签名和单步增量签名使用它们自己的列车分割统计数据归一化。签名状态中不会附加提示标签、语言标记、夹具命令历史记录、未来操作或分支标识符。

表 9: TRACE 签名使用的机器人状态通道。

Channel block	Dim.	Units	Normalisation for signature input	Zeroed channel rule
Base planar velocity (v_x, v_y)	2	m/s	Train mean/std after the zero mask with std floor 10^{-6}	Fixed-base tasks set both channels to 0
Base yaw velocity ω	1	rad/s	Train mean/std after the zero mask with std floor 10^{-6}	Fixed-base tasks set this channel to 0
Left arm joints $q_{0,\dots,6}^L$	7	rad	Train mean/std, then append to the history path	Never zeroed
Right arm joints $q_{0,\dots,6}^R$	7	rad	Train mean/std, then append to the history path	Never zeroed
Full state path $s_{1,\dots,t}$	17	normalised units	Piecewise linear path over masked, standardised states	Constant zero channels add no increments
Path signature ξ_t	5,219	signature units	Train mean/std over history signatures, then $g_t = \phi_g(\xi_t)$	Scalar coordinate omitted
Delta signature δ_t	5,219	signature units	Train mean/std over one-step differences, then $\Delta g_t = \phi_\Delta(\delta_t)$	$\delta_1 = \mathbf{0}$

状态表阐明了哪些内容可以输入签名，哪些内容不能输入签名。路由密钥仅使用屏蔽和标准化的机器人状态路径，恒定的固定基通道不贡献路径增量。不附加提示标签、分支标识符、未来操作或语言标记。这使内存条件保持因果关系，并防止签名成为隐藏的分支线索。

表 10 列出了报告运行中使用的共享 TRACE 设置。这些值定义签名接口、内存宽度和适配器维度。下游政策损失和行动方向不变。

表 10: 核心 TRACE 超参数。

Quantity	Reported setting	Notes
Signature depth p	3	Standard streamed signature
Raw signature dim. d_{sig}	5,219	$17 + 17^2 + 17^3$, scalar term omitted
Signature embedding dims.	512 for g_t , 512 for Δg_t	Shared by both adapters
Slot count K	4 or 6	Chosen by task horizon and branch diversity
Slot width d	512	Matches policy conditioning width
History budget L	24 or 32	Used for training-time causal history reconstruction
History stride r	4 to 8	Covers the recent tail at 30 FPS data rate
Routing hidden size	512	Used for route query and key maps
Adapter hidden size	512	Used by regression memory tokens and diffusion conditioning maps

这些超参数使共享 TRACE 更新程序在两个策略系列中保持相同。深度 3 签名提供路由历史密钥，512-D 内存插槽匹配策略调节宽度，固定历史预算控制监督训练扫描。该表还将这些内存设置与基本策略丢失和操作解码器分开，这些设置保持不变。

A.5 签名深度和不变性

令 S_t 为截至时间 t 的因果状态历史的分段线性插值。TRACE 使用截断的路径签名及其第一个差异

$$\xi_t = \text{Sig}_{\leq p}(S_t) \in \mathbb{R}^{d_{\text{sig}}}, \quad \delta_t = \xi_t - \xi_{t-1}, \quad g_t = \phi_g(\xi_t), \quad \Delta g_t = \phi_\Delta(\delta_t). \quad (13)$$

初始增量为 $\delta_1 = \mathbf{0}$ 。学习到的映射将确定性签名向量转化为路由特征。

对于增加的时间变化 α 和恒定的状态偏移 b ，路由键使用

$$\text{Sig}_{\leq p}(S \circ \alpha) = \text{Sig}_{\leq p}(S), \quad \text{Sig}_{\leq p}^{S_0+b}(S+b) = \text{Sig}_{\leq p}^{S_0}(S). \quad (14)$$

第二个身份使用基点签名。因此，该地址取决于执行的历史几何图形，而不是采样率、执行速度或恒定的坐标偏移。地址仍然对顺序敏感，因此颠倒因果顺序会更改密钥。

对于维度为 d_s 的状态路径和省略标量坐标的标准截断签名，

$$d_{\text{sig}}(p) = \sum_{k=1}^p d_s^k. \quad (15)$$

维度随着深度呈指数增长。通过我们实验中使用的 17-D 状态路径，从 $p=3$ 到 $p=4$ 的跳转将原始签名从 5,219 坐标增加到学习压缩之前的 88,740 坐标。

表 11: $d_s=17$ 的签名深度缩放。

Depth p	Standard d_{sig}	Log signature dim.	Used in reported runs	Practical interpretation
1	17	17	No	Captures net displacement only
2	306	153	No	Adds pairwise order terms at low cost
3	5,219	1,785	Yes	Captures route and phase interactions while remaining practical for learned compression
4	88,740	22,593	No	Requires aggressive compression before routing

我们使用 $p=3$ 作为实际平衡。深度 1 接近位移描述符，并且丢失了许多区分延迟分支的顺序。深度 2 更便宜，但可能不足以代表多阶段历史，例如提示、传输和分支设置。深度 4 大幅增加了特征存储和学习压缩成本，使压缩成为主要的 TRACE 术语。

日志签名很有吸引力，因为它们消除了代数冗余并减少了特征维度，如表 11 所示。作为学习映射的输入，它们的冗余也更少。权衡在于工程和数值复杂性。流式在线更新、增量功能、标准化统计数据和 GPU 支持必须与部署路径匹配。因此，我们在所有报告的实验中使用标准流签名。日志签名是一种注重效率的替代方案，但不属于这些结果的范围。

A.6 诊断指标定义

对于任务 q 和历史转换 T 中的每个保留的在线评估推出 e ，我们沿着转换后的历史运行因果诊断传递，并将其与同一在线推出上的身份传递进行比较。令 I 表示恒等变换， $t_b(e)$ 表示第一个不明确的分支点，而 $\mathcal{P}_e = \{t : t < t_b(e)\}$ 表示分支点之前的历史索引。诊断输入是 $t \in \mathcal{P}_e$ 的槽路由分布 $\omega_t^{T,e} \in \Delta^{K-1}$ 、分支点读数 $z_{t_b(e)}^{T,e} \in \mathbb{R}^d$ 以及由动作头引起的分支动作标签 $\hat{b}^{T,e}$ 。该变换仅应用于生成 TRACE 签名的机器人状态路径。策略使用的转出标识、分支框架、分支标签和视觉观察来自在线执行并且未更改，因此比较隔离了转换后的历史路由的影响，同时保留了在线转出数据源。

路由相似性衡量转换后的历史记录是否通过与分支决策之前的身份在线转出传递相同的内存插槽进行写入。对于具有推出 $\mathcal{E}_q$ 的任务 q ，原始路线得分为

$$r(e, T) = |\mathcal{P}_e|^{-1} \sum_{t \in \mathcal{P}_e} \frac{\langle \omega_t^{T,e}, \omega_t^{I,e} \rangle}{\|\omega_t^{T,e}\|_2 \|\omega_t^{I,e}\|_2}, \quad (16)$$

$$R_q(T) = |\mathcal{E}_q|^{-1} \sum_{e \in \mathcal{E}_q} r(e, T). \quad (17)$$

路由向量是非负概率分布，因此余弦相似度位于 $[0, 1]$ 。高路由相似性意味着转换后的历史记录遵循与原始 Full TRACE 部署相同的软槽地址序列。报告的值被标准化为同一任务上的 Full TRACE 身份在线部署传递，

$$\text{RouteSim}_q(T) = 100 \frac{R_q(T)}{R_q(I)}. \quad (18)$$

对于 Full TRACE 参考，构造为 $R_q(I) = 1$ 。我们保持分母明确，因为在对任务进行平均之前，所有任务级别分数都是相对于该参考进行报告的。

分支一致性衡量从内存中读取的分支点证据是否被保留以及是否支持相同的延迟分支动作。我们计算连续读出项和离散动作协议项，

$$C_q^{\text{read}}(T) = |\mathcal{E}_q|^{-1} \sum_{e \in \mathcal{E}_q} \frac{1 + \frac{\langle z_{t_b(e)}^{T,e}, z_{t_b(e)}^{I,e} \rangle}{\|z_{t_b(e)}^{T,e}\|_2 \|z_{t_b(e)}^{I,e}\|_2}}{2}, \quad (19)$$

$$C_q^{\text{act}}(T) = |\mathcal{E}_q|^{-1} \sum_{e \in \mathcal{E}_q} \mathbf{1}[\hat{b}^{T,e} = \hat{b}^{I,e}], \quad (20)$$

$$B_q(T) = \frac{1}{2} (C_q^{\text{read}}(T) + C_q^{\text{act}}(T)), \quad (21)$$

$$\text{BranchCons}_q(T) = 100 \frac{B_q(T)}{B_q(I)}. \quad (22)$$

读出余弦从 $[-1, 1]$ 映射到 $[0, 1]$ ，动作项是与原始 Full TRACE 分支动作的一致率。高分支一致性意味着转换后的历史在第一个不明确的决策中暴露出类似的内存读数，并导致操作头选择相同的分支。最终表值是任务平衡平均值， $|\mathcal{Q}|^{-1} \sum_q$ 指标 $_q(T)$ 。标准误差是通过在每个任务中的展开进行引导重采样并重新计算任务平衡平均值来计算的。

顺序反转被用作负控制，因为它保留了保留的情节、分支框架和许多边缘状态值，但它破坏了历史的因果顺序。这正是路径签名和槽路由密钥要编码的信息。时间重采样或速度抖动等保序变换使两种诊断保持较高水平，而当 TRACE 使用有序历史证据而不是仅在决策点附近或没有顺序的相同状态时可见的线索时，反转会降低诊断水平。完整 TRACE 是身份在线部署参考，因此构造时将 I 的两个归一化诊断报告为 100。这些数量仅在训练后计算。它们不是损失，不用于模型选择，并且不通过它们获取梯度。

A.7 长期内存稳定性

上述不变性诊断测试分支决策是否依赖于有序历史证据。我们还运行长期历史稳定性检查，以检查固定槽内存是否仍然可用，因为保留的在线推出在分支读出之前提供了更长的因果历史记录。此过程是对报告范围的诊断，而不是对任意情节长度的理论保证。

该协议有两种形式。在报告范围内的在线历史通行证中，每个保留的物理展示都提供了因果历史。经过训练的更新程序沿着执行历史应用，并且内存状态记录在第一个不明确的分支点。在扩展历史诊断中，我们继续进行相同的在线历史积累，并通过记录的在线部署和部署的诊断通道来测量因果部分。当这些分段在记录的诊断源中可用时，扩展会使用重复的干扰分段、速度抖动、稀疏重采样和其他共享路由在线分段。分支框架、分支标签、目视观察、

并且策略权重保持固定，因此诊断分数的变化反映了记忆路径和存储的历史证据的变化。这些诊断行并不是推断的推出成功率。

为了长期历史稳定性，我们记录槽变动、写入熵、槽占用、分支读出一致性和分支决策一致性。对于具有写入分布 $\omega_1, \dots, \omega_T$ 的剧集，这些是

$$\text{Churn} = (T - 1)^{-1} \sum_{t=2}^T \mathbf{1} \left[\arg \max_j \omega_t^{(j)} \neq \arg \max_j \omega_{t-1}^{(j)} \right], \quad (23)$$

$$\text{WriteEnt} = (T \log K)^{-1} \sum_{t=1}^T \left(- \sum_{j=1}^K \omega_t^{(j)} \log \omega_t^{(j)} \right), \quad (24)$$

$$\text{Occupancy} = K^{-1} \sum_{j=1}^K \mathbf{1} \left[\exists t \leq T, j = \arg \max_{j'} \omega_t^{(j')} \right]. \quad (25)$$

分支读数一致性是来自扩展历史的分支读数与来自匹配的报告范围在线历史通道的读数之间的余弦相似度。分支决策的一致性就是相应动作的一致率。

表 12: 固定插槽 TRACE 内存的长期内存稳定性诊断。价值观是任务平衡的手段而不是推迟的部署。读数和决策一致性将每个扩展与匹配的报告范围分支读数进行比较。

Diagnostic pass	History extension	Slot churn↓	Write entropy↓	Slot occupancy↑	Readout cons.↑	Decision cons.↑
Reported-horizon online history	1.0×	0.010	0.356	0.646	1.000	1.000
Extended history	1.25×	0.017	0.374	0.690	0.982	0.968
Extended history	1.5×	0.025	0.397	0.710	0.962	0.936
Extended history	2.0×	0.041	0.431	0.744	0.928	0.904
Repeated distractor extension	1.5×	0.052	0.462	0.758	0.907	0.872
Shared-route online extension	2.0×	0.035	0.412	0.737	0.943	0.916

稳定性读数是评估的扩展的测量诊断，而不是任意范围的证明。在评估的扩展中，时隙流失保持在 0.05 附近或更低，占用率保持广泛但未崩溃，即使在重复的干扰片段下，分支决策一致性也保持至少 0.872。干扰器扩展下较大的熵来自于添加的片段对不明确的共享路径观测的重用，而分支读数仍然接近报告的视野参考。上升的搅动或熵与下降的分支一致性一起是覆盖或扩散写入失败的特征。因此，诊断仅支持评估范围和历史扩展范围内的内存声明。它没有表明固定时隙可以避免覆盖任意长的部署的旧信息。

A.8 适配器架构

两种 TRACE 变体都使用相同的签名索引内存更新器。正文通过其基本目标“回归”和“扩散”来命名这两个变体。在本架构小节中，相同的变体被描述为回归适配器和扩散适配器，因为图和表指的是向每个基本策略呈现内存读数的面向策略的模块。对于按 e 索引的策略系列，

$$H_t = (M_t, z_t^{\text{mem}}, g_t, \Delta g_t), \quad \mathcal{A}_\theta^{(e)} : \mathcal{H}_{\text{TRACE}} \rightarrow \mathcal{C}_e, \quad c_t^{(e)} = \mathcal{A}_\theta^{(e)}(H_t). \quad (26)$$

这里 $\mathcal{C}_e$ 是下游策略族的本地调节空间。适配器改变策略输入条件，但保持动作参数化和模仿目标不变。

表 13: 回归和扩散主干的适配器接口。

Adapter	TRACE inputs	Adapter computation	Expert conditioning produced	Expert loss
Regression adapter	Slot memory $M_t \in \mathbb{R}^{K \times 512}$, readout z_t^{mem} , signature embeddings $g_t, \Delta g_t$	Map each slot with W_M . Map $[z_t^{\text{mem}}, g_t, \Delta g_t]$ into one summary token. Optionally use the summary for feature modulation	512-D memory tokens concatenated to the regression policy attention memory. The action head regresses the native action chunk	Unchanged chunk L1 loss plus the standard optional KL term
Diffusion adapter	Same $M_t, z_t^{\text{mem}}, g_t, \Delta g_t$ from the shared updater	Pool slots with readout attention weights. Concatenate $\text{pool}(M_t), z_t^{\text{mem}}, g_t$, and Δg_t . Pass the result through a zero-initialised MLP	512-D additive global conditioning vector appended to time-step and action conditioning. Diffusion denoising iterations are unchanged	Unchanged diffusion denoising loss over the action trajectory
Shared updater	Masked state path, pooled multi-view visual feature v_t , proprioceptive embedding p_t , and previous slots M_{t-1}	Compute $g_t, \Delta g_t$. Route a write distribution ω_t . Update gated slot candidates and read slots with the current visual-state query	Policy-agnostic TRACE memory state H_t consumed by either adapter	Auxiliary balance, entropy, and consistency losses only during training

适配器比较显示了两个 TRACE 变体的不同之处。两者消耗相同的路由记忆状态，但回归适配器将其公开为注意力记忆标记，而扩散适配器将其公开为全局条件向量。这就是为什么主要实验可以将共享收益归因于因果记忆，同时仍然允许每个政策系列保留自己的行动损失。

表 14 将未更改的基本策略前向传递与 TRACE 的每步计算分开。条目为平均/p95 延迟。回归行是在具有 CUDA 12.8 和 PyTorch 2.9.1 的 NVIDIA GeForce RTX 5090 上通过流回归策略配置文件测量超过 1,900 个预热后控制步骤。扩散行将测量的共享 TRACE 路径与部署的 TRACE 条件策略中记录的扩散和适配器时序相结合。

表 14: 每步部署延迟细分（以毫秒为单位）。

Policy path	Base forward	Signature	Slot update	Readout	Adapter	TRACE overhead	Total loop	Overhead
Regression adapter	5.90 / 7.21	0.52 / 0.69	0.90 / 1.26	0.58 / 0.74	0.10 / 0.15	2.11 / 2.66	8.32 / 10.02	35.8%
Diffusion adapter [†]	118.00 / 142.00	0.52 / 0.69	0.90 / 1.26	0.58 / 0.74	0.14 / 0.21	2.15 / 2.72	120.46 / 144.87	1.8%

[†] 扩散计时使用共享的测量 TRACE 路径。Base Forward 是 100 步扩散去噪调用。

延迟表显示 TRACE 在未更改的基本策略调用之前添加了一个小的固定计算。对于回归适配器，开销是可见的，因为基本策略已经很快。对于扩散适配器来说，与 100 步扩散去噪调用相比，相同的内存路径较小。因此，该表支持 TRACE 是在线调节模块而不是第二次策略传递或离线检索过程的主张。

A.9 内存更新、训练和在线推理

在每一步中，相机特征被汇集为 $v_t = C^{-1} \sum_c \rho(f_{\text{vis}}(o_t^{(c)}))$ ，本体感受嵌入为 $p_t = \phi_s(\tilde{s}_t)$ 。这些特征形成了视觉状态内容 $e_t = [v_t, p_t]$ 。地址特征 q_t 是根据投影路径签名和增量签名构建的。当启用第一帧锚定时， q_t 还包括因果第一帧锚定。有了这个地址向量，路由查询和写分布就是

$$\rho_t = \phi_\rho(q_t), \quad \bar{m}_{t-1,k} = m_{t-1,k} + \lambda_\eta \eta_k, \quad (27)$$

$$\ell_{t,k} = \frac{(W_q \rho_t)^\top W_k \bar{m}_{t-1,k}}{\tau \sqrt{d_r}}, \quad \omega_{t,k} = \frac{\exp(\ell_{t,k})}{\sum_{j=1}^K \exp(\ell_{t,j})}. \quad (28)$$

其中， η_k 是启用时的固定正弦槽标识；设置 $\lambda_\eta = 0$ 给出不使用插槽标识的适配器。内容提案、写力、槽位更新为

$$u_t = \phi_w(e_t, q_t), \quad (29)$$

$$\tilde{m}_{t,k} = \tanh(\phi_m(\bar{m}_{t-1,k}, u_t, \rho_t)), \quad (30)$$

$$\beta_{t,k} = \omega_{t,k} \sigma(\phi_\beta(\bar{m}_{t-1,k}, u_t, \rho_t)), \quad (31)$$

$$m_{t,k} = (1 - \beta_{t,k}) m_{t-1,k} + \beta_{t,k} \tilde{m}_{t,k}. \quad (32)$$

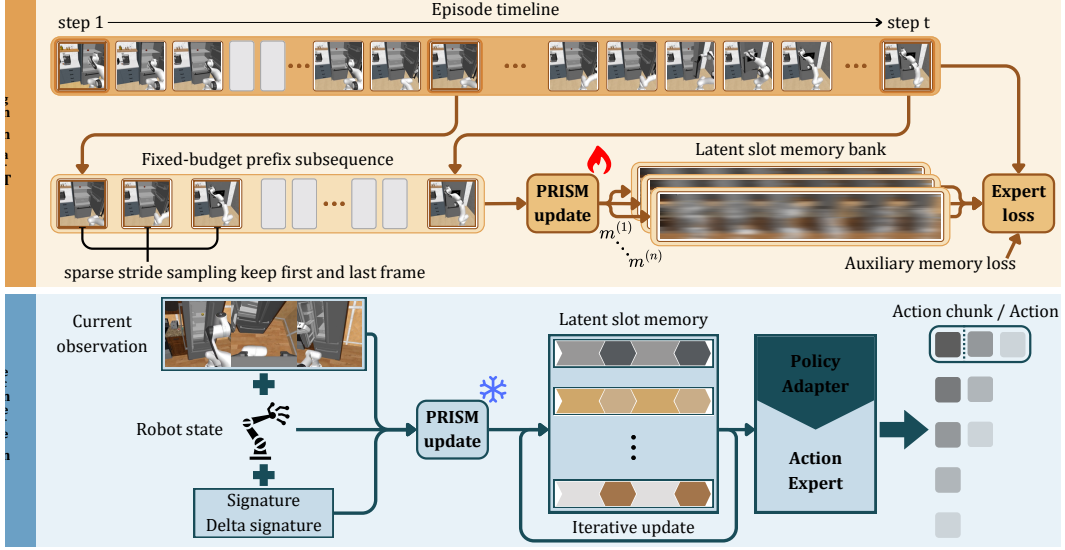


图 5: 训练和推理一致性。训练会扫描在线可用的屏蔽固定预算历史记录，部署会在每个执行步骤应用相同的更新程序。

内存读出使用单独的注意力分布：

$$u_t^r = \phi_r(e_t, \rho_t), \quad (33)$$

$$\bar{m}_{t,k} = m_{t,k} + \lambda_\eta \eta_k, \quad (34)$$

$$a_{t,k} = \frac{(u_t^r)^\top W_k^r \bar{m}_{t,k}}{\sqrt{d}}, \quad (35)$$

$$\alpha_{t,k} = \frac{\exp(a_{t,k})}{\sum_{j=1}^K \exp(a_{t,j})}, \quad (36)$$

$$z_t^{\text{mem}} = \sum_{k=1}^K \alpha_{t,k} W_v^r m_{t,k}. \quad (37)$$

在训练期间，每个目标时间 t 使用带有有效掩码 b_l 的填充固定预算因果历史索引序列 $I_{L,r}(t) = (i_1, \dots, i_L) \subseteq \{1, \dots, t\}$ 。所选的有效条目包含第一个可用帧、当前帧和最近的跨步采样尾部。从 $M^{(0)} = \mathbf{0}$ 开始，扫描为

$$M^{(l)} = \mathcal{U}_\theta(M^{(l-1)}, o_{i_l}, s_{i_l}, q_{i_l}, b_l), \quad l = 1, \dots, L. \quad (38)$$

此扫描应用部署在每个联机步骤中使用的相同更新程序以及缓存的内存状态。最终扫描的内存形成 H_t ，专家族 e 的原生策略损失为

$$\mathcal{L}_{\text{policy}}^{(e)} = \mathbb{E}_{(\tau, t) \sim \mathcal{D}} \left[\ell_e \left(\mathcal{E}_\psi^{(e)}(x_t, \mathcal{A}_\theta^{(e)}(H_t)), y_t^* \right) \right]. \quad (39)$$

有限时隙更新器在训练期间具有三种常见的故障模式。槽位折叠通过几个槽位写入大部分证据。扩散写入将一项观察结果分散到多个槽中，从而使后面的地址变得很弱。读写不匹配以后续更新后读出无法恢复的形式存储信息。TRACE 针对每种情况使用一个轻量级稳定器，然后保持下游仿物镜不变。

$$\mathcal{L} = \mathcal{L}_{\text{policy}}^{(e)} + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}}. \quad (40)$$

对于目标 t 的第 l 个选定历史元素，令 $\omega_{t,l}$ 为槽路由分布， $u_{t,l}$ 为写入建议， $z_{t,l}^{\text{mem}}$ 为内存读出。对于 $N_v = \sum_{t,l} b_{t,l}$ 和 $\bar{\omega}^{(j)} = N_v^{-1} \sum_{t,l} b_{t,l} \omega_{t,l}^{(j)}$,

auxiliary terms are

$$\mathcal{L}_{\text{bal}} = K^{-1} \sum_{j=1}^K \left(\bar{\omega}^{(j)} - K^{-1} \right)^2, \quad (41)$$

$$\mathcal{L}_{\text{ent}} = (N_v \log K)^{-1} \sum_{t,l} b_{t,l} \left(- \sum_{j=1}^K \omega_{t,l}^{(j)} \log \omega_{t,l}^{(j)} \right), \quad (42)$$

$$\mathcal{L}_{\text{cons}} = N_v^{-1} \sum_{t,l} b_{t,l} d^{-1} \left\| \tanh z_{t,l}^{\text{mem}} - \tanh u_{t,l} \right\|_2^2. \quad (43)$$

$\mathcal{L}_{\text{bal}}$ 惩罚不均匀的平均路由，因此它可以防止插槽崩溃。 $\mathcal{L}_{\text{ent}}$ 在每个有效的写入步骤中惩罚高熵路由，因此它不鼓励分散写入。 $\mathcal{L}_{\text{cons}}$ 将有界内存读出与当前写入建议对齐，因此减少了读写不匹配。

这些术语是有限能力的稳定器，而不是理论上的保证。他们偏向于平衡、可寻址和可读写入的训练，但他们并没有证明每条因果信息仍然是可恢复的。由于存储器具有固定数量的槽和有界的槽宽度，因此足够长的视野或轨迹以及比存储器可以表示的更多信息仍然可能迫使多个事实共享容量。在这种情况下，后来的更新可能会覆盖或模糊早期的内容，特别是当证据被重复路由到附近的槽位或当视觉上相似的结果需要不同的解释时。

辅助术语仅作用于记忆。下游策略损失仍然是块 L1，具有用于回归的标准可选 KL 项，以及用于扩散的本机扩散去噪损失。

算法 1 因果 TRACE 内存更新和适配器调节。

要求：观察 o_t 、机器人状态 s_t 、先前记忆 M_{t-1} 、先前签名 ξ_{t-1} 、可选第一帧锚点 a^0 、状态掩码 Z_q 、训练分割归一化统计、策略族 $e \in \{\text{回归}, \text{扩散}\}$ 。确保：更新内存 M_t 、签名 ξ_t 、适配器调节 c_t 和策略预测 $\hat{y}_t$ 。1：应用任务掩码和状态标准化。

$$\bar{s}_t = Z_q(s_t), \quad \tilde{s}_t = (\bar{s}_t - \mu_s) / (\sigma_s + \epsilon).$$

2：将 $\tilde{s}_t$ 附加到分段线性历史路径 S_t 。3：计算 $\xi_t = \text{Sig}_{\leq p}(S_t)$ 和 $\delta_t = \xi_t - \xi_{t-1}$ 。4：去除标量签名坐标，标准化 ξ_t 和 δ_t ，然后将它们映射到 $g_t = \phi_g(\xi_t)$ 和 $\Delta g_t = \phi_\Delta(\delta_t)$ 。5：将当前观测值编码为 $v_t = C^{-1} \sum_c \rho(f_{\text{vis}}(o_t^{(c)}))$ 和 $p_t = \phi_s(\tilde{s}_t)$ 。6：计算地址特征 $q_t = [g_t, \Delta g_t]$ ，仅在启用第一帧锚点路由时附加 a^0 ，并设置 $\rho_t = \phi_\rho(q_t)$ 。7：使用公式 28 路由写入。8：根据视觉状态内容和地址状态形成门控写入候选，然后使用公式 32 更新槽。9：使用公式 37 读取存储器。10：构造 $H_t = (M_t, z_t^{\text{mem}}, g_t, \Delta g_t)$ 和 $c_t^{(e)} = \mathcal{A}_\theta^{(e)}(H_t)$ 。11：使用不变的基本策略预测 $\hat{y}_t = f_\psi(z_t, c_t^{(e)})$ 。

A.10 其他案例研究

表 15 总结了五项任务背后的具体延迟证据结构。这些案例研究将主要论文中的诊断量与推出级行为联系起来，并为正文中讨论的基线故障模式提供证据来源。

表 15: 将基线失败、证据和 TRACE 行为联系起来的其它任务级案例研究。

Task	History evidence	Ambiguous branch	Baseline failure	Failure evidence	TRACE behaviour
Tool	Initial object identity determines the later tool sequence	Object and tool are no longer jointly visible when the sequence must branch	Wrong tool order at the branch, or correct branch followed by contact loss	Table 1 shows high Tool variance, and Table 18 separates delayed-memory errors from contact losses	Stores the object-dependent history and improves branch selection, while contact-rich stages still create variance
Book	Origin shelf determines route and placement	After pickup and transit, the overhead view is similar across origins	Places on a visually plausible target that is inconsistent with the origin route	Figure 4 shows similar views after transit, and Table 2 shows the matched memory gap	Reads the origin-dependent slot near placement and preserves route-specific evidence
Laundry	Origin side determines brush-and-basket versus fold-and-store	Garment pose becomes visually similar after the shared carry segment	Executes the visually dominant routine regardless of origin side	Tables 1 and 2 show the largest gains on this origin-side branch task	Routes the side cue during pickup and reuses it at the later branch
Cable	Origin side determines the matched device	Traversal and local device pose provide evidence still visible near the device choice	Reaches the workspace but can choose the wrong device when local geometry is ambiguous	Tables 1 and 2 show closer scores, consistent with partial evidence from current geometry	Memory helps at the device choice, but local manipulation still contributes many credited stages
Medicine	Tray origin determines box selection and return	Boxes enter a shared bedside area before the return decision	Selects or returns the wrong box after a correct past pickup	Tables 1 and 2 show gains on the tray-origin branch, and Table 3 supports history-sensitive readout	Keeps the tray-origin cue available through the shared staging segment

案例研究解释了为什么相同的内存模块具有不同的任务级效果。Book、Laundry 和 Medicine 的大部分后期分数都放在记住起源线索上，因此 TRACE 改变了主要的分支错误模式。Cable 在器件选择附近仍然包含有用的局部几何形状，这缩小了差距，而 Tool 将延迟分支与接触密集型执行相结合，因此需要附录 A.11 中的单独方差分析。

A.11 Variance and Failure Analysis

表 1 中工具任务的标准误差最高。它的进度分数结合了离散延迟记忆决策和决策后的多个接触丰富的操作阶段。卷展栏可以记住正确的对象和工具分支，但由于抓取滑动、工具接触弱或恢复超时而丢失许多已记入的阶段。错误的延迟分支还可以同时删除多个下游阶段。这解释了为什么“工具”比“书籍”、“洗衣店”和“药品”噪音更大，其中起源提示更直接地决定分支分数。

表 16 报告了对主要平均值的简单稳健性检查。drop-Tool 列删除具有最大 SE 的任务，并从其余四个任务均值中重新计算任务平衡均值。TRACE 回归仍然是最强的方法，为 72.92，TRACE 扩散仍然高于最强的非 TRACE 基线，分别为 59.79 和 51.71。仅使用报告的任务 SE 为 TRACE 回归提供了 [71.28, 74.55] 的 drop-Tool 95% 间隔，为 TRACE 扩散提供了 [57.10, 62.48]，为最强的非 TRACE 基线提供了 [48.13, 55.29]。此检查不使用工具卷展栏，因此高方差任务不会带来平均增益。该表直接根据表 1 计算得出。

表 16: 用于主要比较的留一任务平衡平均值。值根据表 1 中的任务平均值重新计算。

Method	All tasks	Drop Tool	Drop Book	Drop Laundry	Drop Cable	Drop Medicine
Diffusion Policy	25.00	26.75	28.17	25.75	22.75	21.58
ACT	25.50	24.12	28.46	29.00	20.88	25.04
$\pi_{0.5}$	50.47	51.71	50.33	51.21	50.83	48.25
GR00T N1.6	30.60	28.50	35.08	32.25	28.75	28.42
SmolVLA	28.33	29.17	30.42	32.42	22.92	26.75
X-VLA	33.90	41.00	30.62	32.12	33.38	32.38
Ours (Regression)	69.23	72.92	65.79	66.29	73.79	67.38
Ours (Diffusion)	59.53	59.79	57.33	56.79	61.66	62.08

表 17 将嘈杂的工具得分与总体结论分开。工具对于两种 TRACE 变体都有很宽的间隔，因为离散延迟分支后面是几个接触丰富的阶段。同一张表还使用表 1 中报告的任务 SE 报告了 drop-Tool 平均值及其与最强的非 TRACE 基线的差距。回归相对于 $\pi_{0.5}$ 保持了 21.21 点 drop-Tool 优势，而扩散则保持了 8.08 点优势。

表 17: TRACE 变体的工具任务不确定性和丢弃工具稳健性。工具间隔使用报告的平均值 ± 1.96 SE。Drop-Tool 间隔是根据表 1 中报告的任务 SE 计算的。

Method	Tool mean $\pm$ SE	Tool 95% interval	Drop-Tool avg.	Drop-Tool gap vs. $\pi_{0.5}$
Ours (Regression)	54.50 $\pm$ 17.96	[19.30, 89.70]	72.92 [71.28, 74.55]	+21.21 [17.27, 25.15]
Ours (Diffusion)	58.50 $\pm$ 10.17	[38.57, 78.43]	59.79 [57.10, 62.48]	+8.08 [3.60, 12.56]

表 18 给出了用于解释高工具方差的故障类别。这些类别将 TRACE 是否以正确的对象条件决策到达延迟分支与后来的接触丰富损失区分开来。这种区别很重要，因为工具可以出于不同的原因产生中档或低档进度。一些推出保留了延迟的提示，但通过接触失去了下游阶段，而另一些则使依赖于记忆的分支本身失败。

表 18: 用于解释高方差推出的工具故障分类。这些类别将延迟记忆故障与分支后的接触和恢复故障分开。

Failure class	Operational definition	Score pattern	Interpretation
Correct branch with contact loss	The object-dependent tool order is selected correctly, then progress drops from grasp slip, weak contact, or incomplete tool engagement	Shared setup and branch stages receive credit, but later manipulation stages are missing	Memory succeeds, while execution noise lowers total progress
Wrong branch or tool order	The shared setup reaches the delayed decision, but the selected tool sequence does not match the initial object identity	past stages receive credit, followed by a sharp loss at the branch and its downstream stages	Direct delayed-evidence error. This is the failure mode TRACE is designed to reduce
Recovery timeout	Manipulation stalls during recovery after a partial success and exceeds the timeout	Progress plateaus after partial later-stage credit	Separates memory retention from long-horizon recovery and contact robustness
Pre-branch setup failure	The rollout fails before the delayed decision can be evaluated	Low progress before the branch point	Does not test delayed memory, but still contributes to rollout-level Tool variance
Other or unclassified	The score trace or video evidence does not cleanly match the categories above	Mixed or inconsistent stage evidence	Reserved for audit before making branch-level rate claims